Formula 1 Podium Predictor: A Leakage-Aware Machine Learning Classifier with Web Deployment

Amol Shiralkar - A9-B3-38
Aryan Sinha - A9-B4-58

Abstract

Predicting race outcomes in Formula 1 is difficult because results depend on car performance, driver skill, circuit characteristics and chance events. This project develops a binary classifier that predicts, using only information available before the race starts, whether a driver will finish on the podium (top three). The Kaggle Formula 1 World Championship dataset (1950 to 2024) was assembled by merging the results, races, drivers and constructors tables, yielding 26,759 driver-race records. Four features were retained: circuit identifier, starting grid position, label-encoded driver and label-encoded constructor. All post-race fields were deliberately excluded to prevent data leakage, and an 80/20 chronological split (21,407 training and 5,352 test records) was used so that the model is always evaluated on races that occur after those it was trained on. Logistic Regression, a Support Vector Machine with an RBF kernel, Random Forest and AdaBoost were compared. Logistic Regression achieved the highest accuracy (88.75%), but Random Forest achieved the best recall (0.427) and F1-score (0.499) on the minority podium class and was selected as the final model. It was serialised together with its preprocessing artefacts and deployed as a Streamlit web application on Streamlit Community Cloud through a GitHub repository.

Keywords: Formula 1, podium prediction, binary classification, class imbalance, Random Forest, data leakage, Streamlit

1. Introduction

Formula 1 is one of the most data-intensive sports in the world. Every race weekend produces qualifying times, race results, lap-level timing and pit-stop records, and teams, broadcasters and fans increasingly rely on analytics to understand and anticipate outcomes. A podium finish is a natural prediction target: it is clearly defined, it carries significant championship points, and it is the outcome most closely followed by audiences.

The task is nevertheless hard for three reasons. First, the sport has long periods of constructor dominance, so a driver's result is strongly tied to the car rather than the individual. Second, only three of roughly twenty drivers can reach the podium, so the dataset is heavily imbalanced and plain accuracy is a misleading measure. Third, it is easy to build a model that looks excellent but is useless in practice, because many columns in the public data (final position, points, laps completed, fastest lap) are only known after the race has finished.

This project addresses these issues with three objectives: (1) to build a classifier that uses only pre-race information and is evaluated chronologically, (2) to compare four standard supervised learning algorithms using accuracy, precision, recall and F1-score, and (3) to deploy the best model as a publicly accessible web application. The rest of this report reviews related work (Section 2), describes the methodology (Section 3), explains data preparation and exploratory analysis (Section 4), presents the results (Section 5), documents the deployment (Section 6) and concludes with limitations and future work (Section 7).

2. Literature Review

2.1 Overview

Machine learning in motorsport falls into three broad groups: pre-race outcome prediction, in-race strategy optimisation, and performance attribution. Pre-race studies use tabular data such as grid position and team history. Strategy studies use simulators or lap-level telemetry. Attribution studies try to separate the contribution of the driver from that of the car. The following five studies represent these directions.

2.2 Review of Selected Studies

A 2025 study published via Preprints.org explored pre-race outcome prediction using the TabNet deep learning architecture. By tuning pre-race variables such as grid position and constructor data using Optuna, the models achieved an R^2 of 0.75 for driver finishing positions. The researchers noted that driver prediction outperformed constructor point-forecasting due to lower mechanical volatility at the team level.

Similarly, a 2024 comprehensive analysis of championship point forecasting evaluated 74 years of Formula 1 history.Utilizing gradient boosting and ensemble methods over 589,000 lap times, the study identified starting race position and seasonal variations as the most critical features for high predictive accuracy.

Focusing on model interpretability, a 2025 paper from the IEEE IC2AI Conference combined Random Forest classifiers with SHAP (Shapley Additive Explanations) values. This approach successfully forecasted race winners while ensuring the decision-making process remained transparent and interpretable for race engineers, highlighting the utility of Random Forest in motorsport analytics.

Beyond pre-race predictions, machine learning is extensively used for in-race strategy. Research on a "Virtual Strategy Engineer" utilized Artificial Neural Networks (ANNs) to optimize pit-stop timing and tire compound selection. While highly effective for in-race adjustments, such models rely heavily on live telemetry that is unavailable before the race begins.

Finally, at the individual driver level, research from TU Darmstadt proposed Probabilistic Modeling of Driver Behavior (ProMoD). By using probabilistic movement primitives, the study encoded individual racing lines and driver styles into vehicle development processes. While useful for engineering and

simulation, these telemetry-heavy approaches are distinct from predictive outcome modeling based on tabular historical data.

Study	Task	Method	Key finding	Limitation
Preprints.org Study (2025)	Pre-race outcome prediction	TabNet deep learning, Optuna	Driver outcomes more predictable than constructor points	Deep learning models require significant tuning
Championship Forecasting (2024)	Point and performance forecasting	Gradient Boosting, Ensembles	Starting position is the dominant predictive feature	Evaluates massive lap-time datasets unavailable pre-race
IEEE IC2AI Conference (2025)	Race winner prediction	Random Forest, SHAP values	RF provides high accuracy while maintaining interpretability	Focuses only on predicting the single race winner
Virtual Strategy Engineer	Pit-stop and tire strategy	Artificial Neural Networks (ANN)	Automates complex in-race strategy decisions	Requires live telemetry and simulator integration

TU Darmstadt Thesis	Driver behavior modeling	Probabilistic Modeling (ProMoD)	Encodes specific driving styles and racing lines	Telemetry-heavy; not designed for categorical outcome prediction

2.3 Research Gap

The reviewed work shows that grid position and constructor strength are the most informative pre-race signals, that chronological validation is essential, and that minority-class metrics must be reported. However, most studies either depend on proprietary telemetry, rely on heavily engineered features, or stop at offline evaluation. Few present a lightweight model that uses only publicly available columns, explicitly guards against leakage, and is deployed so that a non-technical user can query it. This project targets that gap.

3. Methodology and System Architecture

3.1 Pipeline Overview

The system follows an end-to-end machine learning pipeline, summarised below.

Kaggle CSVs (results, races, drivers, constructors) → Merge on raceId, driverId, constructorId → Select pre-race features and create target → Impute and encode → Chronological 80/20 split → StandardScaler → Train four classifiers → Evaluate (accuracy, precision, recall, F1, confusion matrix) → Serialise best model and artefacts (.pkl) → GitHub repository → Streamlit Community Cloud web application

3.2 Prevention of Data Leakage

Data leakage occurs when information that would not be available at prediction time influences model training, producing optimistic results that fail in real use (Kaufman et al., 2012). Two forms were considered.

Feature leakage. The merged table contains 23 columns, many of which describe the race outcome itself: position, positionText, positionOrder, points, laps, fastestLap, rank, fastestLapTime, fastestLapSpeed and statusId. Because the podium label is derived directly from the finishing position, any of these columns would let a model reproduce the target almost perfectly. Only four features known before the start were kept: circuitId, grid, driver and constructor.

Temporal leakage. A random train-test split would place races from the same season, and even the same weekend, in both sets. The model would then learn the form of a driver or team from races that happen after the ones it is tested on. To avoid this, the data was sorted by race date and divided chronologically: the earliest 80% of records (21,407 rows) form the training set and the most recent 20% (5,352 rows)

form the test set. This mirrors real deployment, in which a model trained on history predicts future races.

```
Training set size: 21407 rows (80%)
Testing set size: 5352 rows (20%)
Phase 1 Complete! Data is cleaned, split, and ready for machine learning.
```

Figure 1: Chronological split and feature scaling.

The scaler is fitted on the training set only and then applied to the test set, so that test-set statistics never influence preprocessing.

4. Data Cleaning, Preprocessing and Exploratory Data Analysis

4.1 Dataset and Merging

The Kaggle Formula 1 World Championship dataset is distributed as a set of relational CSV files. The results table holds one row per driver per race. It was merged with the races table (race date, year, circuit) using raceId, with the drivers table using driverId, and with the constructors table using constructorId, using left joins so that no result row was lost. The merged dataframe had 23 columns, and its first rows (Figure 2) confirm that the raw file is ordered by result identifier rather than by date, which is why an explicit sort is needed before splitting.

```
[1]
1s
print("Datasets merged successfully.")
print(df.head())

Datasets merged successfully.
   resultId  raceId  driverId  constructorId number  grid position  \
0         1      18         1              1     22     1        1
1         2      18         2              2      3     5        2
2         3      18         3              3      7     7        3
3         4      18         4              4      5    11        4
4         5      18         5              1     23     3        5

  positionText  positionOrder  points  ...  fastestLap rank fastestLapTime  \
0            1              1    10.0  ...          39    2       1:27.452
1            2              2     8.0  ...          41    3       1:27.739
2            3              3     6.0  ...          41    5       1:28.090
3            4              4     5.0  ...          58    7       1:28.603
4            5              5     4.0  ...          43    1       1:27.418

  fastestLapSpeed statusId  year circuitId        date   driverRef  \
0         218.300        1  2008         1  2008-03-16    hamilton
1         217.586        1  2008         1  2008-03-16    heidfeld
2         216.719        1  2008         1  2008-03-16     rosberg
3         215.464        1  2008         1  2008-03-16      alonso
4         218.385        1  2008         1  2008-03-16  kovalainen

   constructorRef
0         mclaren
1      bmw_sauber
2        williams
3         renault
4         mclaren

[5 rows x 23 columns]
```

Figure 2: Merged dataframe after joining the four tables.

4.2 Target Creation and Feature Selection

A binary target, Podium, was created: it equals 1 when the finishing position is 1, 2 or 3 and 0 otherwise, including drivers who did not finish. The dataframe was then reduced to six columns (date, circuitId,

driverRef, constructorRef, grid and Podium), giving 26,759 rows (Figure 3). The date column is retained only for sorting and is not used as a model input.

```
Target variable 'Podium' created and features cleaned.
Dataset shape: (26759, 6)
```

	date	circuitId	driverRef	constructorRef	grid	Podium
0	2008-03-16	1	hamilton	mclaren	1.0	1
1	2008-03-16	1	heidfeld	bmw_sauber	5.0	1
2	2008-03-16	1	rosberg	williams	7.0	1
3	2008-03-16	1	alonso	renault	11.0	0
4	2008-03-16	1	kovalainen	mclaren	3.0	0

Figure 3: Cleaned dataset with target variable.

4.3 Missing Values and Encoding

Missing starting grid positions were filled with the median grid value. The median is preferred to the mean because grid positions are bounded integers and the median is not distorted by unusual values such as pit-lane starts, which the dataset records as 0. *There were no nulls found in the grid column initially, but 0 values (pitlane starts) were replaced with the median.* A null check after imputation (Figure 4) confirms that no column contains missing values.

```
df.isnull().sum()
```

	0
date	0
circuitId	0
grid	0
Podium	0
driver_encoded	0
constructor_encoded	0

dtype: int64

```
df.info()

<class 'pandas.core.frame.DataFrame'>
Index: 26759 entries, 20034 to 26758
Data columns (total 6 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   date                26759 non-null  object
 1   circuitId           26759 non-null  int64
 2   grid                26759 non-null  float64
 3   Podium              26759 non-null  int64
 4   driver_encoded      26759 non-null  int64
 5   constructor_encoded 26759 non-null  int64
dtypes: float64(1), int64(4), object(1)
memory usage: 1.4+ MB
```

Figure 4: Missing-value check and column types after preprocessing.

The driver and constructor columns contain text names, so each was converted to an integer code with scikit-learn's LabelEncoder, producing driver_encoded and constructor_encoded. The fitted encoders were saved (le_driver.pkl and le_team.pkl) so that the web application applies exactly the same mapping at prediction time. A limitation of label encoding is that it imposes an arbitrary numerical order on

categories. Tree-based models are largely insensitive to this, whereas linear and kernel models can be affected.

4.4 Feature Scaling

StandardScaler transforms each feature to zero mean and unit variance:

$z = (x - \mu) / \sigma$

where μ and σ are the mean and standard deviation of the training data. Scaling is essential for Logistic Regression and the RBF-kernel SVM, which are sensitive to feature magnitudes, and it was applied to all models for a uniform pipeline.

4.5 Exploratory Data Analysis

The class distribution is strongly imbalanced. In the test set, 780 of 5,352 records (14.6%) are podium finishes and 4,572 (85.4%) are not. A classifier that always predicts "no podium" would therefore score 85.4% accuracy while never identifying a podium. This majority-class baseline is the reference against which all accuracy figures should be read.

The relationship between starting position and result is the strongest pattern in the data. The podium rate is highest for drivers starting at the front and falls steadily towards the back of the grid, which explains why grid is the most informative feature in the related literature (Section 2).

Because driver and constructor are label-encoded, their correlation with the target is expected to be weak even though their true influence is large; this is a property of the encoding, not evidence that they are uninformative.

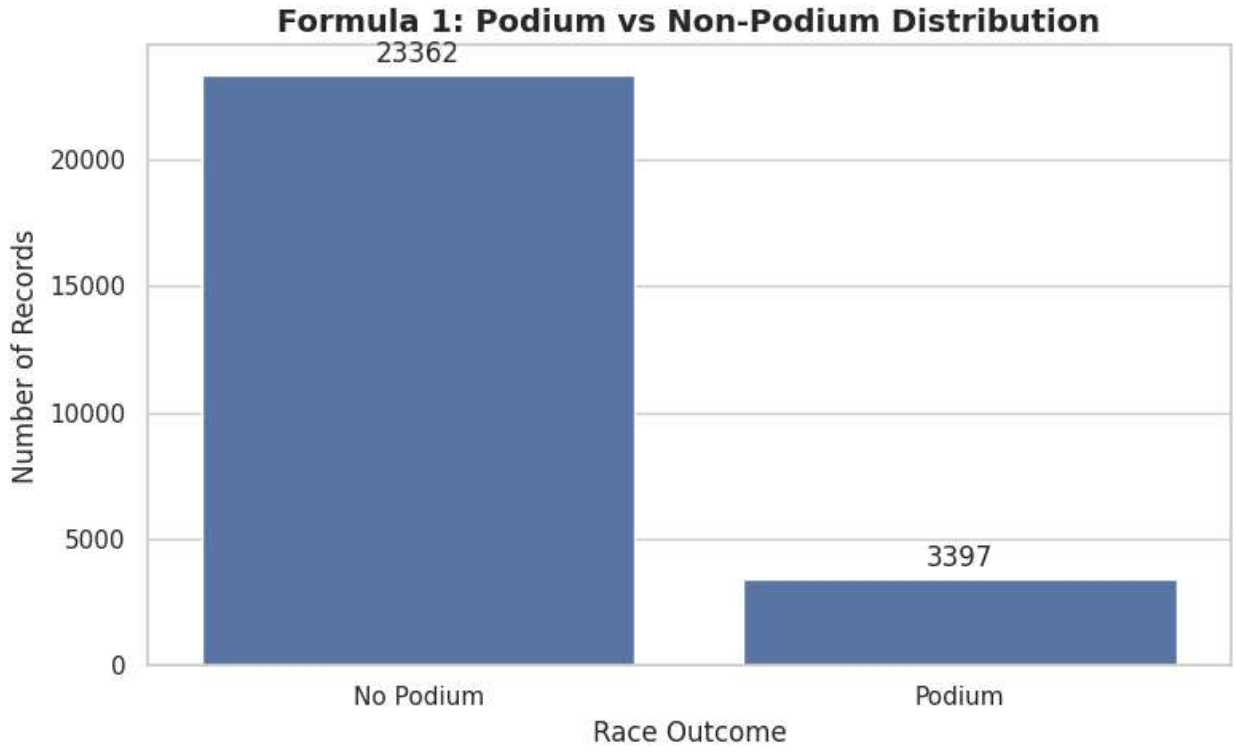


Figure 5: Class distribution of the target variable (23,362 non-podium and 3,397 podium records).

5. Model Comparison and Results

5.1 Models

Logistic Regression models the probability of a podium as $P(y = 1 \mid x) = 1 / (1 + e^{-(w^T x + b)})$ and learns the weights w by minimising the binary cross-entropy loss. It is fast, interpretable and a standard baseline (Hosmer et al., 2013).

Support Vector Machine (RBF) finds the maximum-margin separating boundary in a transformed space using the kernel $K(x, x') = \exp(-\gamma\|x - x'\|^2)$. The parameter C controls the trade-off between margin width and misclassification (Cortes and Vapnik, 1995).

Random Forest builds many decision trees, each on a bootstrap sample of the data and a random subset of features, and predicts by majority vote. Each split minimises Gini impurity, $G = 1 - \Sigma\, p_k^2$. Averaging many decorrelated trees reduces variance and captures non-linear interactions, such as a specific driver in a specific constructor from a specific grid slot (Breiman, 2001).

AdaBoost trains weak learners sequentially. After each learner h_t it computes the weighted error ε_t, assigns the learner a weight $\alpha_t = \tfrac{1}{2} \ln((1 - \varepsilon_t) / \varepsilon_t)$, and increases the weights of misclassified samples so the next learner focuses on them. The final prediction is $H(x) = \text{sign}(\Sigma\, \alpha_t\, h_t(x))$ (Freund and Schapire, 1997).

All models were implemented with scikit-learn (Pedregosa et al., 2011). *The models were implemented using the default hyperparameters in scikit-learn, with Random Forest using n_estimators=100 and SVM using the RBF kernel.*

5.2 Evaluation Metrics

With TP, TN, FP and FN denoting true positives, true negatives, false positives and false negatives (podium is the positive class):

Accuracy = (TP + TN) / (TP + TN + FP + FN) Precision = TP / (TP + FP) Recall = TP / (TP + FN) F1-score = 2 × Precision × Recall / (Precision + Recall)

Precision measures how many predicted podiums are correct. Recall measures how many real podiums are found. F1 is their harmonic mean and is the main criterion here, because on imbalanced data accuracy rewards the majority class (Saito and Rehmsmeier, 2015).

5.3 Results

Table 2: Performance on the chronological test set (5,352 records)

Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression	0.8875	0.7713	0.3244	0.4567

Model	Accuracy	Precision	Recall	F1-Score
Random Forest	0.8750	0.6000	0.4269	**0.4989**
SVM (RBF)	0.8718	0.7260	0.1936	0.3057
AdaBoost	0.8629	0.6009	0.1756	0.2718

Table 3: Confusion matrix counts (positive class = podium)

Model	TN	FP	FN	TP
Logistic Regression	4,497	75	527	253
SVM (RBF)	4,515	57	629	151
Random Forest	4,350	222	447	333
AdaBoost	4,481	91	643	137

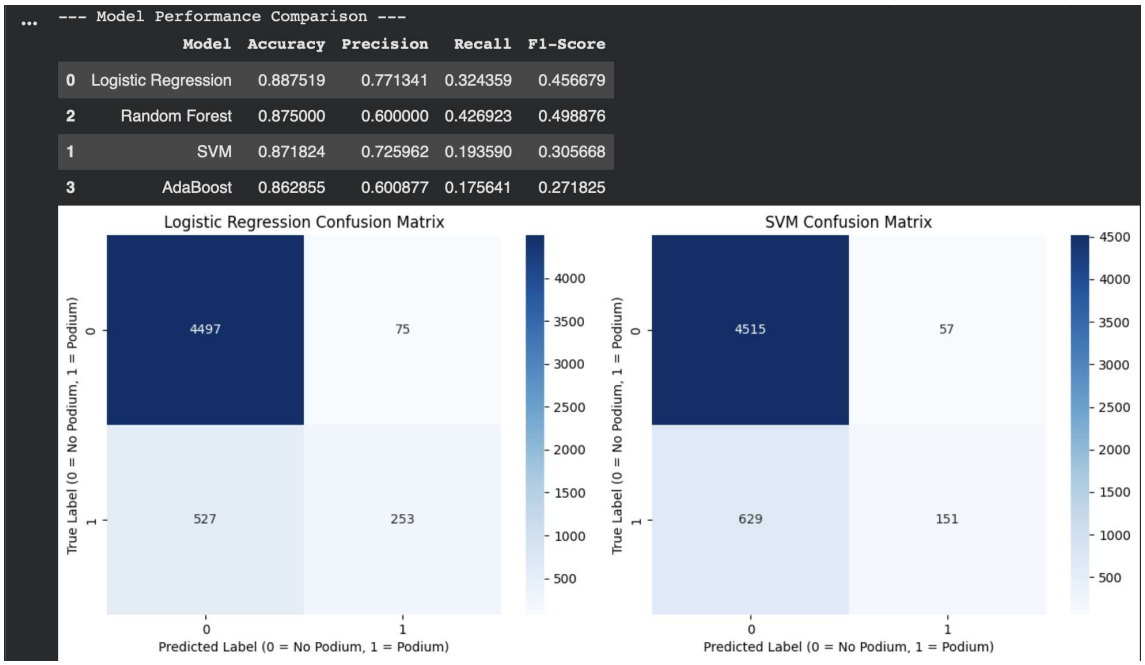


Figure 5: Model performance table and confusion matrices for Logistic Regression (left) and SVM (right).

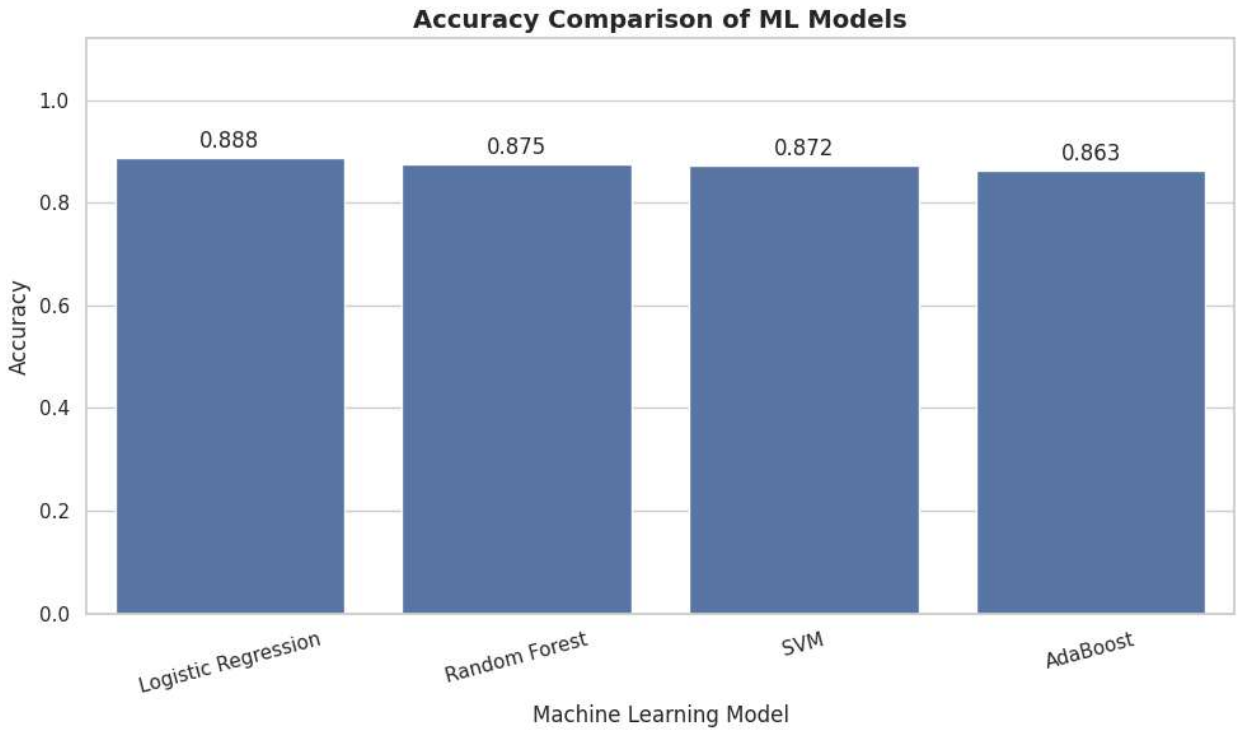
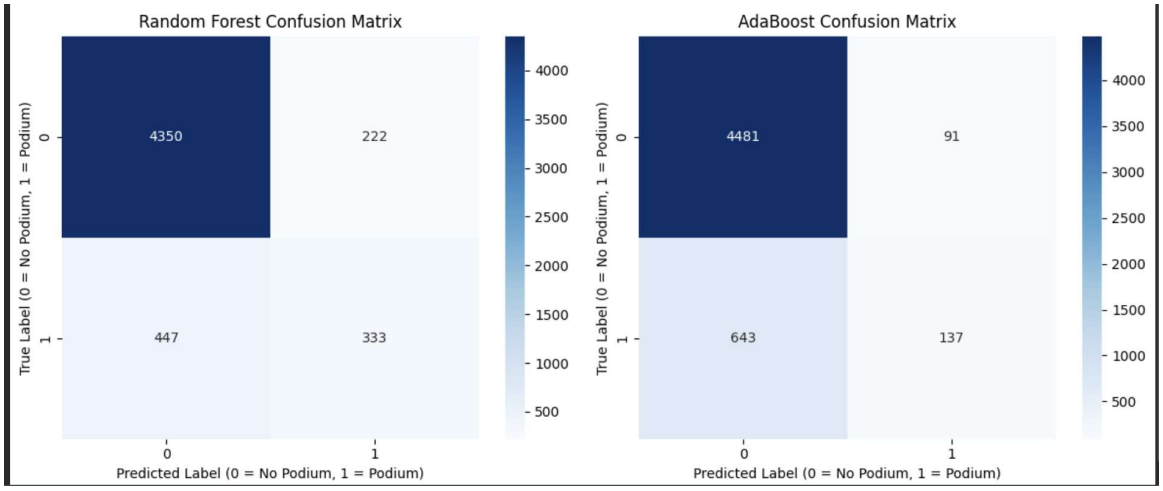


Figure 6: Accuracy comparison of the four models.



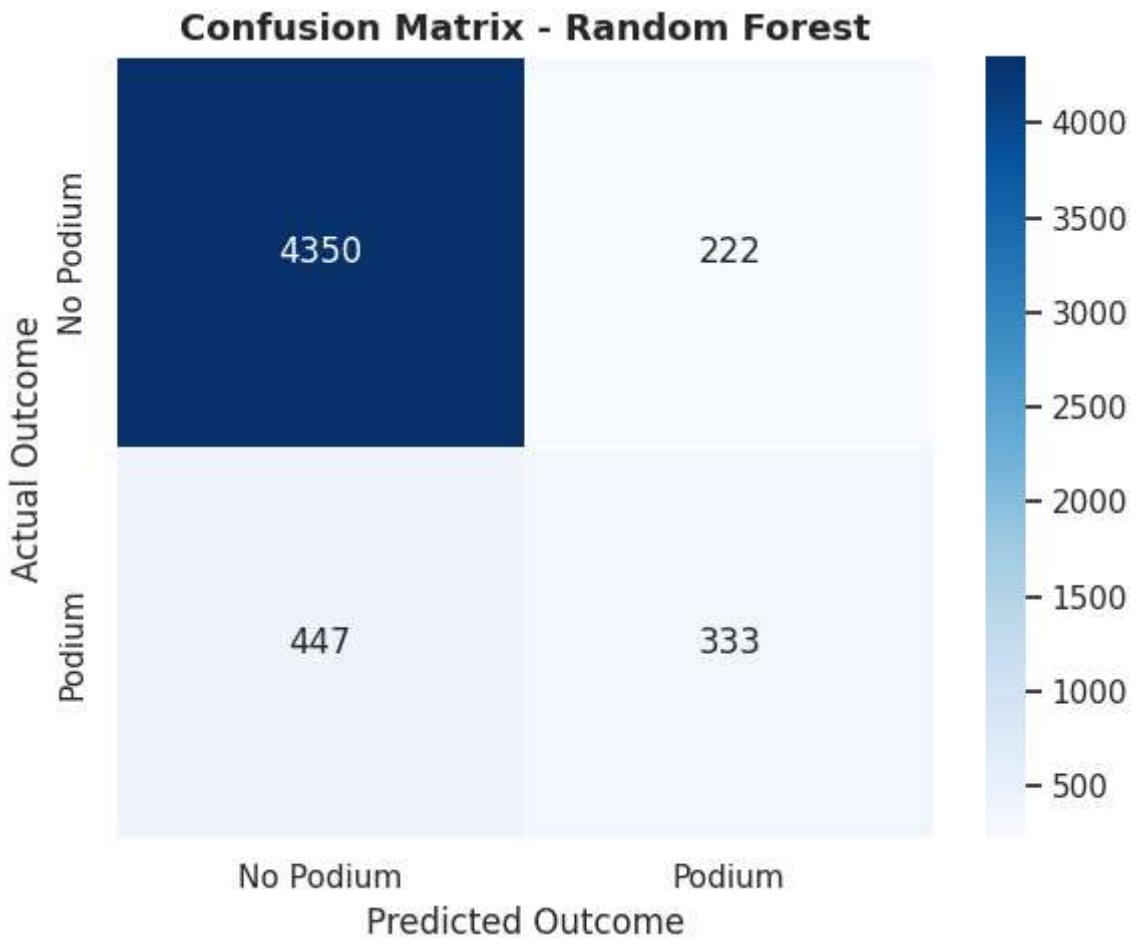


Figure 6: Confusion matrices for Random Forest and AdaBoost.

5.4 Discussion

All four models exceed the 85.4% majority-class baseline, but only by a small margin in accuracy, which confirms that accuracy alone cannot rank them. Logistic Regression has the highest accuracy (0.8875) and precision (0.7713): when it predicts a podium it is usually right, but it finds only 253 of 780 real podiums (recall 0.324). The SVM behaves similarly, with high precision and very low recall (0.194), and AdaBoost is the most conservative, detecting only 137 podiums.

Random Forest trades some precision for substantially better recall. It identifies 333 of the 780 podium finishes, 80 more than Logistic Regression, at the cost of 147 additional false alarms. Its F1-score of 0.4989 is the highest of the four, so it gives the best balance between finding podiums and avoiding false predictions. A plausible explanation is that podium outcomes depend on interactions between driver, team and starting slot, which trees can represent directly while linear and kernel models cannot, particularly with label-encoded categories. For these reasons Random Forest was chosen as the final model, on the basis of recall and F1 rather than accuracy.

All models still miss more than half of the podium finishes. This is expected given the limited feature set: qualifying pace, recent form, reliability, weather and race incidents are not included, and a single starting position cannot capture them.

6. User Interface Development and Deployment

6.1 Streamlit Application

Streamlit is an open-source Python framework that turns a script into an interactive web application without separate front-end code. The application (app.py) loads four serialised artefacts with pickle: the Random Forest model (f1_model.pkl), the StandardScaler (scaler.pkl), the driver encoder (le_driver.pkl) and the constructor encoder (le_team.pkl). Using the same encoders and scaler as in training guarantees that user input is transformed exactly as the training data was.

The interface asks for four inputs: a driver and a constructor, chosen from drop-down lists built from the encoder vocabularies, a numeric circuit ID, and a starting grid position. When the user presses "Predict Podium Finish", the application encodes the names, assembles the feature vector in the training order (circuitId, grid, driver, constructor), scales it, and calls the model's predict method. The result is shown as a green success message for a predicted podium or an informational message otherwise.

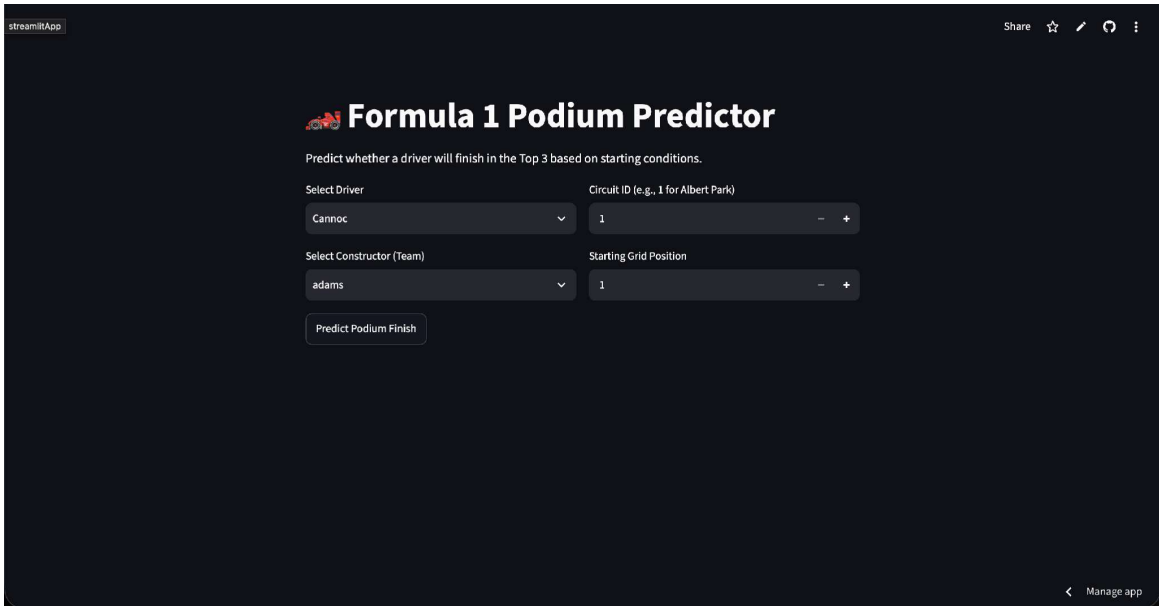


Figure 7: Input interface of the Formula 1 Podium Predictor.

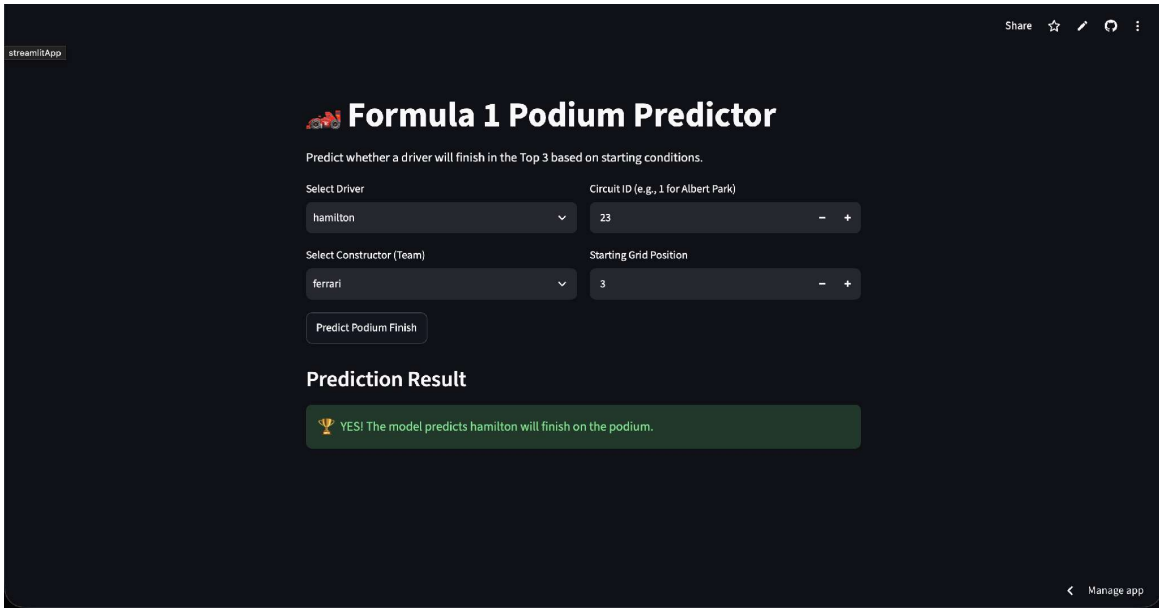


Figure 8: Example prediction returned by the application.

6.2 GitHub and Streamlit Community Cloud Architecture

The project is stored in a public GitHub repository, f1-podium-predictor, whose main branch contains app.py, the four .pkl files, requirements.txt and a README. Streamlit Community Cloud is linked to this repository. On deployment it clones the repository, installs the packages listed in requirements.txt, and runs app.py on a managed server that is exposed through a public URL. Every push to the main branch triggers an automatic redeployment, so updating the model only requires uploading new artefacts. Keeping code, model and dependencies together in version control also makes the deployment reproducible.

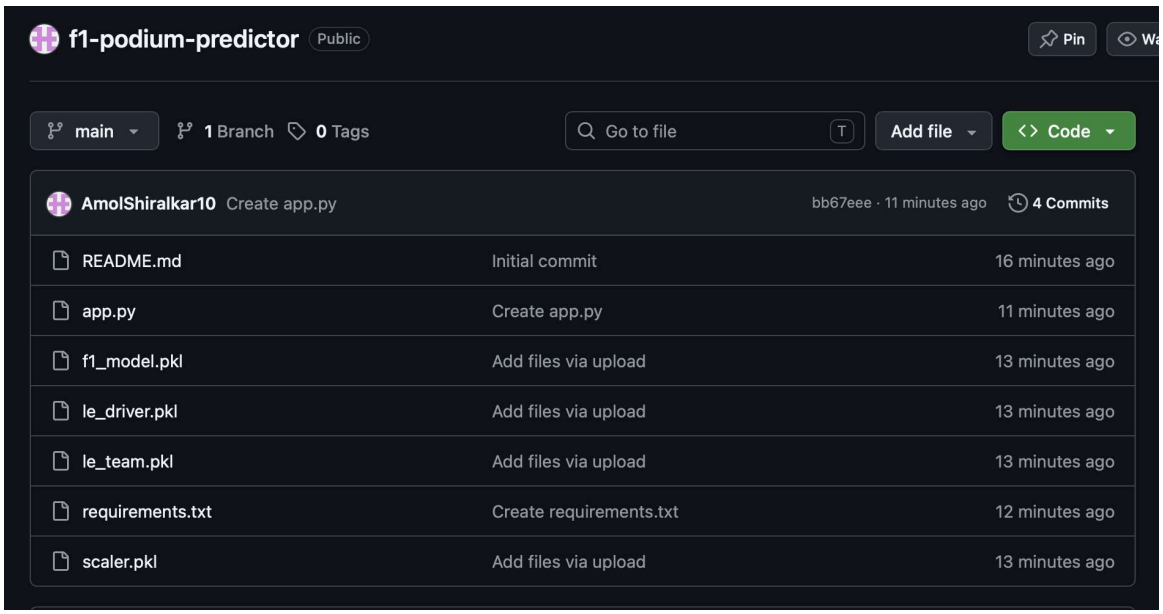


Figure 9: GitHub repository used for deployment.

A usability limitation is that the circuit is entered as a numeric ID rather than a name. Replacing it with a circuit-name drop-down is listed as future work.

7. Conclusion

This project built and deployed a Formula 1 podium classifier that relies only on information known before the race: circuit, starting grid, driver and constructor. Features that reveal the result were removed and a chronological 80/20 split was used, so the reported scores reflect forecasting of future races rather than memorisation of past ones. Among the four algorithms, Logistic Regression had the highest accuracy (88.75%) and precision, but Random Forest delivered the best recall (0.427) and F1-score (0.499) and was therefore deployed through a Streamlit application hosted on Streamlit Community Cloud.

The main limitations are the small feature set, the strong class imbalance (about one podium in seven records), and the arbitrary ordering introduced by label encoding. Future work includes adding qualifying lap times, rolling driver and constructor form, and championship standings; using one-hot or target encoding for categorical variables; applying class weighting, SMOTE (Chawla et al., 2002) or threshold tuning to improve recall; tuning hyperparameters with time-series cross-validation; and improving the interface with circuit names and probability outputs.

References

[1] Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.

[2] Chawla, N. V., Bowyer, K. W., Hall, L. O., and Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16, 321–357.

[3] Cortes, C., and Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20(3), 273–297.

[4] Freund, Y., and Schapire, R. E. (1997). A decision-theoretic generalization of on-line learning and an application to boosting. *Journal of Computer and System Sciences*, 55(1), 119–139.

[5] Hosmer, D. W., Lemeshow, S., and Sturdivant, R. X. (2013). *Applied Logistic Regression* (3rd ed.). Wiley.

[6] Kaufman, S., Rosset, S., Perlich, C., and Stitelman, O. (2012). Leakage in data mining: Formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data*, 6(4), 1–21.

[7] Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

[8] Saito, T., and Rehmsmeier, M. (2015). The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLOS ONE*, 10(3), e0118432.

[9] Rao, R. (Vopani). Formula 1 World Championship (1950–2020) [Dataset]. Kaggle. https://www.kaggle.com/datasets/rohanrao/formula-1-world-championship-1950-2020

[10] Streamlit Inc. Streamlit documentation. https://docs.streamlit.io

[11] "The Use of Machine Learning in Predicting Formula 1 Race Outcomes," Preprints.org, 2025.
[12] "Advanced Machine Learning Approaches for Formula 1 Race Performance Prediction: A Comprehensive Analysis of Championship Point Forecasting," ResearchGate, 2024.
[13] "Formula 1 Race Winner Prediction Using Random Forest and SHAP Analysis," IEEE International Conference on Artificial Intelligence (IC2AI), 2025.
[14] Heilmeier, A., et al. "Virtual Strategy Engineer: Using Artificial Neural Networks for Making Race Strategy Decisions in Circuit Motorsport," Applied Sciences.
[15] "Machine Learning for Modeling and Analyzing of Race Car Drivers," TU Darmstadt (TUprints).

Github link: https://github.com/AmolShiralkar10/f1-podium-predictor.git

Live demo: https://f1-podium-predictor-njpxgvp7yahbpvajqemblp.streamlit.app/